

PurpleForce Gym

Sistema de Gestión Integral de Gimnasio

MEMORIA DEL PROYECTO INTERMODULAR

Autor: Daniel García Docío

Ciclo / Curso: 2º DAM — 2025/26

Centro: Nortempo Formación

Fecha de entrega: 8 de junio de 2026

Exposición: 12 de junio de 2026

Tecnologías principales

Java 17 · Spring Boot 3.2 · Spring Security 6 · MySQL 8 · Thymeleaf · OpenPDF · JUnit 5

Resumen (Abstract)

PurpleForce Gym es una aplicación web completa para la gestión integral de un gimnasio. Está dirigida a tres tipos de usuarios: administradores, instructores y socios. El sistema centraliza la consulta y reserva de clases, la gestión de entrenadores y tipos de actividad, el control de aforo en tiempo real y la generación de informes exportables en PDF y CSV.

Las funcionalidades clave incluyen: autenticación segura con tres roles diferenciados (ADMIN, INSTRUCTOR, SOCIO), reserva de clases con control de concurrencia transaccional, ranking de clases más populares basado en consultas JPQL avanzadas, exportación de informes PDF personalizados y gestión completa del catálogo de ejercicios y entrenadores.

1. Motivación e introducción

1.1 Motivación

La gestión de un gimnasio implica coordinar horarios, entrenadores, aforos y reservas de manera simultánea. En muchos centros pequeños y medianos esta tarea se realiza con hojas de cálculo, llamadas telefónicas o aplicaciones genéricas que no cubren las necesidades específicas del sector. PurpleForce Gym nace con el objetivo de proporcionar una solución web integral, accesible desde cualquier dispositivo, que automatice y centralice toda la operativa del gimnasio.

1.2 Contexto y problema a resolver

Antes de la aplicación, un socio debía llamar al gimnasio para reservar una clase, sin saber en tiempo real cuántas plazas quedaban disponibles. El administrador llevaba un registro manual de las reservas, con el riesgo de sobre-reservas y errores. Los instructores no disponían de forma digital de consultar su agenda ni la lista de alumnos inscritos en cada sesión.

PurpleForce Gym resuelve todos estos problemas: el socio reserva desde la web en segundos, el sistema controla el aforo automáticamente evitando sobre-reservas mediante transacciones en base de datos, y tanto el administrador como el instructor disponen de un panel centralizado con toda la información actualizada en tiempo real.

1.3 Objetivos propuestos

Objetivo general:

- Desarrollar una aplicación web funcional y completa que cubra los requisitos técnicos del módulo Proyecto Intermodular de 2º DAM.

Objetivos específicos:

- Implementar autenticación segura con tres roles de usuario distintos (ADMIN, INSTRUCTOR, SOCIO).
- Gestionar el ciclo completo de reservas con control de aforo en tiempo real.
- Desarrollar consultas JPQL no triviales: ranking de popularidad e historial por usuario.
- Generar informes exportables en PDF (historial de socio, horario de instructor) y CSV (ranking).
- Garantizar el manejo concurrente de usuarios mediante transacciones y sesiones independientes.
- Documentar el proyecto con pruebas funcionales automatizadas y memoria técnica completa.

2. Metodología, recursos y planificación

2.1 Metodología utilizada

Se ha seguido una metodología iterativa e incremental, dividiendo el proyecto en sprints semanales. Se empezó por la capa de persistencia y seguridad, añadiendo funcionalidades de manera progresiva. Se han realizado commits frecuentes en GitHub para mantener un histórico claro del avance del proyecto. Se utilizó Postman para probar los endpoints manualmente durante el desarrollo, antes de escribir los tests automáticos con JUnit 5.

2.2 Estimación de recursos

Hardware:

- PC de desarrollo: procesador Intel Core i5, 16 GB RAM, SSD 512 GB.
- Servidor local: la aplicación se ejecuta en localhost:8080.

Software:

- Visual Studio Code — IDE principal de desarrollo.
- MySQL 8.0 + MySQL Workbench — base de datos relacional y cliente gráfico.
- Git + GitHub — control de versiones y repositorio remoto.
- Postman — pruebas manuales de endpoints HTTP.
- Maven 3.8 — gestión de dependencias y construcción del proyecto.

2.3 Planificación (hitos y cronograma)

Semana / Fechas	Hito	Tareas principales
1-2 (09-20 abr)	Análisis y planificación	Definir requisitos, modelo de datos, diagrama ER
3-4 (21 abr-04 may)	Configuración y entidades	Setup Spring Boot, entidades JPA, SecurityConfig

Semana / Fechas	Hito	Tareas principales
5-6 (05-18 may)	Backend principal	Repositorios, servicios, controladores Admin y Auth
7 (19-25 may)	Panel Socio e Instructor	Reservas, filtros, historial, panel instructor
8 (26 may-01 jun)	Informes y exportaciones	Generación PDF (OpenPDF) y CSV (Apache Commons)
9 (02-08 jun)	Pruebas y correcciones	10 pruebas JUnit, corrección de bugs
10 (09-12 jun)	Memoria y presentación	Redacción memoria, preparar demo para la expo

3. Tecnologías y herramientas

Tecnología	Categoría	Uso en el proyecto
Java 17	Lenguaje	Backend / lógica de negocio
Spring Boot 3.2	Framework	Gestión del servidor, rutas e inyección de dependencias
Spring Security 6	Seguridad	Autenticación por formulario, roles, BCrypt, sesiones
Spring Data JPA	Persistencia	Acceso a BD con repositorios JPA; soporte JPQL avanzado
Thymeleaf 3.1	Motor plantillas	Renderizado HTML en servidor con integración Spring MVC
MySQL 8.0	Base de datos	BD relacional para almacenar todos los datos del sistema
H2 (tests)	BD tests	Base de datos en memoria para ejecución de JUnit
Bootstrap Icons	UI / iconos	Librería de iconos SVG para la interfaz web
OpenPDF 1.3	Informes PDF	Exportación de historial de reservas y horarios en PDF
Apache Commons CSV	Exportación	Generación del ranking de clases en formato CSV
Lombok	Código	Generación automática de getters, setters y builders
JUnit 5 + MockMvc	Testing	Pruebas funcionales con contexto completo de Spring
Maven 3.8	Build	Gestión del proyecto y sus dependencias
VS Code	IDE	Entorno de desarrollo principal
Git + GitHub	Versiones	Seguimiento de cambios y repositorio remoto
Postman	Pruebas API	Pruebas manuales de endpoints HTTP durante desarrollo

4. Análisis

4.1 Definición de requisitos

Requisitos funcionales

ID	Nombre	Descripción
RF-01	Login con roles	Autenticación con tres roles: ADMIN, INSTRUCTOR, SOCIO
RF-02	Registro de socios	Un nuevo usuario puede crear su cuenta como socio
RF-03	Gestión de clases	El admin puede crear, editar y cancelar clases
RF-04	Gestión entrenadores	El admin puede crear, editar y desactivar entrenadores
RF-05	Tipos de clase	El admin define tipos con precio, nivel y duración
RF-06	Catálogo ejercicios	El admin gestiona el catálogo completo de ejercicios
RF-07	Reservar clase	Un socio reserva plaza en una clase con disponibilidad
RF-08	Cancelar reserva	Un socio cancela una reserva confirmada previamente
RF-09	Historial de reservas	El socio ve su historial completo de reservas con estado
RF-10	Ranking de clases	Clases ordenadas por número de reservas (consulta no trivial 1)
RF-11	Historial por usuario	Reservas filtradas y ordenadas por usuario (consulta no trivial 2)
RF-12	Exportar PDF	El socio descarga su historial PDF; el instructor su horario PDF
RF-13	Exportar CSV	El admin exporta el ranking de clases en formato CSV
RF-14	Ver alumnos	El instructor ve los socios inscritos en cada clase suya
RF-15	Filtros de búsqueda	Las clases se pueden filtrar por nivel y fecha combinados

Requisitos no funcionales

ID	Categoría	Descripción
RNF-01	Seguridad	Contraseñas cifradas con BCrypt; roles protegidos con Spring Security
RNF-02	Rendimiento	Respuesta < 1 segundo en operaciones CRUD normales
RNF-03	Usabilidad	Interfaz responsiva con mensajes de error comprensibles
RNF-04	Portabilidad	Ejecutable en local con Java 17 y MySQL 8; instrucciones claras
RNF-05	Mantenibilidad	Código organizado en capas (MVC) con comentarios y pruebas
RNF-06	Concurrencia	Soporte de sesiones múltiples simultáneas; transacciones ACID

4.2 Modelo de datos

La aplicación utiliza una base de datos relacional MySQL con seis entidades principales. Las relaciones más importantes son: **Clase** tiene una relación N:1 con TipoClase y N:1 con Entrenador. **Reserva** tiene relaciones N:1 con Clase y N:1 con Usuario, formando la tabla de intersección central del sistema.

Entidad	Atributos principales	Descripción
usuario	id, nombre, apellidos, email, password, rol, activo, fechaNac	Persona con acceso al sistema
entrenador	id, nombre, apellidos, especialidad, telefono, email, activo	Imparte las clases del gimnasio
tipo_clase	id, nombre, descripcion, nivel, duracionMinutos, precio	Modalidad de clase (Yoga, HIIT...)
clase	id, tipo_clase_id, entrenador_id, fecha, horaInicio, horaFin, capacidadMaxima, activa	Sesión con fecha y hora
ejercicio	id, nombre, descripcion, grupoMuscular, dificultad, ejerciciosRepetición	Ejercicio dentro del gimnasio
reserva	id, clase_id, usuario_id, fechaReserva, estado (CONFIRMADA, CANCELADA)	Reserva de una clase

Justificación de la base de datos elegida:

MySQL es la opción más adecuada para este proyecto porque los datos del gimnasio son altamente relacionales: una reserva siempre pertenece a un usuario y a una clase concreta, y una clase siempre tiene asignado un entrenador y un tipo. Las relaciones N:1 y N:M se modelan con naturalidad en un esquema relacional. Spring Data JPA ofrece integración nativa con MySQL y permite escribir consultas JPQL avanzadas de forma limpia y mantenible. MySQL garantiza la integridad referencial mediante claves foráneas, lo que es crítico para el control de aforo. Finalmente, MySQL es ampliamente conocido, gratuito y bien documentado, lo que facilita el mantenimiento futuro.

4.3 Casos de uso

ID	Actor	Nombre	Descripción
CU-01	SOCIO	Registrarse	El socio crea su cuenta con email y contraseña
CU-02	TODOS	Iniciar sesión	El usuario se autentica y accede a su panel
CU-03	SOCIO	Ver catálogo clases	El socio visualiza clases con filtros por nivel y fecha
CU-04	SOCIO	Reservar clase	El socio reserva plaza en clase con disponibilidad
CU-05	SOCIO	Cancelar reserva	El socio cancela una reserva confirmada
CU-06	SOCIO	Ver historial	El socio consulta su historial completo de reservas
CU-07	SOCIO	Descargar PDF	El socio exporta su historial en formato PDF

ID	Actor	Nombre	Descripción
CU-08	SOCIO	Ver ranking	El socio consulta las clases más populares
CU-09	INSTRUCTOR	Ver mis clases	El instructor consulta sus clases asignadas
CU-10	INSTRUCTOR	Ver alumnos	El instructor ve los socios inscritos en cada clase
CU-11	INSTRUCTOR	Descargar horario PDF	El instructor exporta su horario en PDF
CU-12	ADMIN	Gestionar clases	El admin crea, edita y cancela sesiones de clase
CU-13	ADMIN	Gestionar entrenadores	El admin da de alta, edita y desactiva entrenadores
CU-14	ADMIN	Gestionar tipos clase	El admin define modalidades de clase con precio
CU-15	ADMIN	Gestionar ejercicios	El admin mantiene el catálogo de ejercicios
CU-16	ADMIN	Gestionar usuarios	El admin crea usuarios con cualquier rol
CU-17	ADMIN	Ver informes	El admin consulta estadísticas y rankings
CU-18	ADMIN	Exportar CSV	El admin descarga el ranking de clases en CSV

5. Diseño

5.1 Diseño general / arquitectura

La aplicación sigue una arquitectura cliente-servidor en tres capas, implementada mediante el patrón MVC de Spring Boot:

- **Capa de presentación (View):** plantillas Thymeleaf + HTML/CSS con tema morado/blanco/negro. Se renderizan en el servidor y se envían al navegador del cliente.
- **Capa de negocio (Controller + Service):** controladores Spring MVC gestionan las peticiones HTTP y delegan la lógica a los servicios. Los servicios aplican reglas de negocio (control de aforo, validaciones, generación de informes).
- **Capa de persistencia (Repository + Model):** entidades JPA mapeadas a tablas MySQL, con repositorios Spring Data que ofrecen CRUD automático y consultas JPQL personalizadas.

El flujo completo de una petición es: *Navegador* → *Dispatcher Servlet* → *Controller* → *Service* → *Repository* → *MySQL* → *Thymeleaf* → *HTML al navegador*.

5.2 Diagrama de clases (UML)

Las clases principales del dominio y sus relaciones:

- **Usuario** (id, nombre, apellidos, email, password, rol: Rol, activo, fechaAlta) → tiene List<Reserva>
- **Entrenador** (id, nombre, apellidos, especialidad, telefono, email, activo) → tiene List<Clase>
- **TipoClase** (id, nombre, descripcion, nivel, duracionMinutos, precio, color) → tiene List<Clase>
- **Clase** (id, tipoClase, entrenador, fecha, horaInicio, horaFin, aforoMaximo, sala, activa) → Métodos: getPlazasOcupadas(), getPlazasLibres(), isDisponible()
- **Reserva** (id, clase, usuario, fechaReserva, estado) — tabla central del sistema de reservas
- **Ejercicio** (id, nombre, descripcion, grupoMuscular, dificultad, materialNecesario)
- **Enum Rol**: ADMIN, INSTRUCTOR, SOCIO

Relaciones: Clase N:1 TipoClase, Clase N:1 Entrenador, Reserva N:1 Clase, Reserva N:1 Usuario.

5.3 Mockups / capturas de interfaz y navegación

Flujo de navegación principal por rol:

- **Cualquier usuario**: /auth/login → /auth/registro (público)
- **Admin**: /admin/dashboard → Clases → Tipos de clase → Entrenadores → Ejercicios → Usuarios → Informes
- **Socio**: /socio/dashboard → Catálogo clases (filtros) → Mis Reservas → Ranking
- **Instructor**: /instructor/dashboard → Mis Clases → Alumnos por clase → Horario PDF

La interfaz utiliza barra de navegación con gradiente morado, tarjetas de estadísticas en el dashboard, tablas con cabeceras moradas, cards para el catálogo de clases con barras de aforo visuales, y formularios con validación en tiempo real.

5.4 Decisiones de diseño importantes

- **1. Control de concurrencia en reservas**: Se usa @Transactional en ReservaService.reservar(). Cuando dos socios intentan reservar la última plaza simultáneamente, la BD garantiza mediante el aislamiento de transacciones que solo una tendrá éxito.
 - **2. Separación de Usuario y Entrenador**: Se separaron para que la tabla de entrenadores almacene datos de contacto específicos sin mezclarlos con credenciales de autenticación.
 - **3. Redirección por rol tras login**: El successHandler de Spring Security redirige automáticamente al dashboard correspondiente según el rol.
 - **4. PasswordConfig separado de SecurityConfig**: Se extrajo el bean PasswordEncoder a una clase propia para evitar la referencia circular entre SecurityConfig y UsuarioService.
-

6. Implementación

6.1 Estructura del proyecto

- `com.purpleforce.gym.model` — Entidades JPA: Usuario, Entrenador, TipoClase, Clase, Ejercicio, Reserva, Rol
- `com.purpleforce.gym.repository` — Interfaces `JpaRepository` con consultas JPQL personalizadas (`@Query`)
- `com.purpleforce.gym.service` — Lógica de negocio: `UsuarioService`, `ClaseService`, `ReservaService`, `InformeService`...
- `com.purpleforce.gym.controller` — `AdminController`, `SocioController`, `InstructorController`, `AuthController`
- `com.purpleforce.gym.config` — `SecurityConfig` (rutas y login) + `PasswordConfig` (BCrypt)
- `src/main/resources/templates/` — Plantillas Thymeleaf en carpetas `admin/`, `socio/`, `instructor/` y `auth/`
- `src/main/resources/static/css/style.css` — Hoja de estilos con variables CSS para el tema morado
- `src/main/resources/data.sql` — Script SQL con datos de ejemplo cargado automáticamente al arrancar

6.2 Componentes principales

Capa de persistencia

Los repositorios extienden `JpaRepository`, lo que proporciona CRUD automático. Se definen consultas JPQL con `@Query` para las operaciones no triviales. El ranking de clases más populares se calcula con un COUNT agrupado por tipo de clase, ordenado descendientemente. El historial de usuario filtra solo reservas CONFIRMADAS y las ordena por fecha de clase descendente.

Capa de servicios

`ReservaService` implementa el control de concurrencia mediante `@Transactional`. El método `reservar()` recarga la clase desde la BD, cuenta las plazas ocupadas con `countConfirmadasByClase()`, y solo si hay plazas disponibles crea la reserva. `InformeService` genera PDF con OpenPDF y CSV con Apache Commons CSVPrinter, ambos en memoria (`ByteArrayOutputStream`).

Seguridad

`SecurityConfig` configura Spring Security 6 con autenticación por formulario. Las rutas `/admin/**` requieren `ROLE_ADMIN`, `/instructor/**` requieren `ROLE_INSTRUCTOR` o `ROLE_ADMIN`, y `/socio/**` requieren `ROLE_SOCIO` o `ROLE_ADMIN`. Las contraseñas se almacenan con `BCryptPasswordEncoder` (factor 10). Se solucionó una referencia circular extrayendo `PasswordEncoder` a `PasswordConfig.java`.

6.3 Dificultades encontradas y soluciones

- **Problema:** referencia circular SecurityConfig ↔ UsuarioService. **Solución:** extraer PasswordEncoder a clase PasswordConfig separada.
 - **Problema:** data.sql fallaba en reinicios por clave duplicada. **Solución:** todos los INSERT usan WHERE NOT EXISTS para ser idempotentes.
 - **Problema:** Spring Security 6 cambió la API. **Solución:** uso del nuevo flujo lambda-based eliminando WebSecurityConfigurerAdapter.
-

7. Multiusuario concurrente

Identificación de usuarios

La autenticación se realiza mediante formulario web gestionado por Spring Security. Al hacer login, Spring crea una sesión HTTP independiente para cada usuario, almacenada en servidor con un JSESSIONID en cookie. Cada sesión mantiene el objeto Authentication del usuario con su email y rol.

Demostración de 2 usuarios simultáneos

Se puede demostrar abriendo dos navegadores distintos (o uno normal y uno en modo incógnito) con credenciales diferentes: por ejemplo, alex@gmail.com (SOCIO) en uno y admin@purpleforce.com (ADMIN) en el otro. La configuración .maximumSessions(5) permite hasta 5 sesiones activas simultáneas.

Control de concurrencia en reservas

El escenario más crítico es que dos socios intenten reservar la última plaza de una clase al mismo tiempo. ReservaService.reservar() está anotado con @Transactional, lo que garantiza que las operaciones de lectura del aforo y creación de la reserva sean atómicas. MySQL aplica aislamiento REPEATABLE READ, asegurando que solo un socio obtenga la plaza.

Fallos de comunicación

Si el servidor no está disponible, Spring Boot devuelve una página de error HTTP estándar. Los formularios incluyen tokens CSRF. Si la sesión expira, Spring Security redirige al login.

8. Despliegue e instalación

8.1 Manual de instalación

Requisitos previos:

- Java 17 o superior instalado (JDK)
- Maven 3.8 o superior
- MySQL 8.0 en ejecución local
- Visual Studio Code con Extension Pack for Java y Spring Boot Extension Pack

Pasos de instalación:

- 1. Clonar el repositorio: `git clone https://github.com/TU_USUARIO/purpleforce-gym.git`
- 2. Crear la BD MySQL: `CREATE DATABASE gimnasio_db CHARACTER SET utf8mb4;`
- 3. Configurar credenciales en `src/main/resources/application.properties`
- 4. Arrancar: abrir `GymApplication.java` y pulsar Run
- 5. Acceder en `http://localhost:8080`

8.2 Configuración

El fichero `application.properties` contiene toda la configuración. Las credenciales de MySQL se configuran en `spring.datasource.username` y `spring.datasource.password`. No se suben contraseñas reales al repositorio.

8.3 Carga de datos de ejemplo

Los datos de ejemplo se cargan automáticamente al arrancar mediante `data.sql`. Incluye 3 entrenadores, 6 tipos de clase, 8 ejercicios, 6 clases programadas y reservas de ejemplo. Las sentencias usan `WHERE NOT EXISTS` para no duplicar datos en reinicios.

8.4 Manual de usuario

Administrador (`admin@purpleforce.com / 1234`): gestionar clases, entrenadores, tipos, ejercicios y usuarios. Apartado Informes para estadísticas y exportar CSV.

Socio (`alex@gmail.com / 1234`): en 'Clases' ver el catálogo con filtros y reservar. En 'Mis Reservas' ver el historial y descargar PDF.

Instructor (`carlos@purpleforce.com / 1234`): ver clases asignadas, consultar alumnos inscritos y descargar el horario en PDF.

9. Pruebas

9.1 Plan de pruebas

El objetivo es verificar que todas las funcionalidades principales responden correctamente tanto en el camino feliz (datos válidos) como en casos de error (credenciales incorrectas, reservas duplicadas, acceso no autorizado). Se implementaron 10 pruebas automáticas con JUnit 5 y MockMvc usando H2 en memoria. Comando: `mvn test`

9.2 Tabla de casos de prueba

ID	Descripción	Procedimiento	Resultado esperado	Estado
PT-01	Registro socio válido	POST /auth/registro datos correctos	Usuario en BD con rol SOCIO	✓ OK
PT-02	Login correcto	POST /auth/login email+pass válidos	Redirección al dashboard	✓ OK
PT-03	Login incorrecto	POST /auth/login contraseña errónea	Redirección con ?error=true	✓ OK
PT-04	Crear tipo de clase	Admin crea tipo 'CrossFit Test'	Tipo guardado en BD	✓ OK
PT-05	Reservar clase	Socio reserva clase con plazas	Reserva con estado CONFIRMADA	✓ OK
PT-06	Reserva duplicada	Socio reserva 2 veces la misma clase	IllegalStateException lanzada	✓ OK
PT-07	Cancelar reserva	Socio cancela su reserva confirmada	Estado CANCELADA en BD	✓ OK
PT-08	Acceso denegado admin	Socio accede a /admin/dashboard	HTTP 403 Forbidden	✓ OK
PT-09	Filtrar por nivel	GET /socio/clases?nivel=PRINCIANTE	Lista de ese nivel	✓ OK
PT-10	Ranking clases	Consulta no trivial de ranking	Lista ordenada desc. por reservas	✓ OK

10. Resultados, conclusiones y vías futuras

10.1 Objetivos alcanzados

- ✓ Autenticación segura con tres roles diferenciados (ADMIN, INSTRUCTOR, SOCIO).
- ✓ CRUD completo de clases, entrenadores, tipos de clase, ejercicios y usuarios.
- ✓ Sistema de reservas con control de aforo en tiempo real y manejo de concurrencia.
- ✓ Dos consultas JPQL no triviales: ranking de popularidad e historial por usuario.
- ✓ Exportación de informes en PDF (historial del socio y horario del instructor).
- ✓ Exportación de ranking de clases en CSV.
- ✓ Datos de ejemplo cargados automáticamente con data.sql al arrancar.
- ✓ 10 pruebas funcionales automatizadas con JUnit 5 y MockMvc.
- ✓ Interfaz responsiva con tema morado/blanco/negro en Thymeleaf.
- ✓ README completo con instrucciones de instalación y ejecución en GitHub.
- — No implementado: gestión de pagos (descartado por alcance del proyecto).
- — No implementado: despliegue en nube (se entrega para ejecución local).

10.2 Conclusiones

El desarrollo de PurpleForce Gym ha permitido consolidar y aplicar de forma práctica los conocimientos adquiridos durante el ciclo de 2º DAM: bases de datos relacionales, programación orientada a objetos con Java, desarrollo web con Spring Boot y seguridad de aplicaciones.

Los aspectos técnicos más relevantes aprendidos han sido: el funcionamiento interno de Spring Security 6 con su nuevo modelo lambda, el control de concurrencia mediante transacciones JPA en escenarios de reserva simultánea, y la generación programática de documentos PDF con OpenPDF.

El resultado es una aplicación funcional, mantenible y bien estructurada que podría adaptarse fácilmente a un gimnasio real con ajustes menores como integrar pagos o desplegarla en la nube.

10.3 Vías futuras / mejoras

- Integración con pasarela de pago (Stripe o PayPal) para suscripciones y clases sueltas.
 - Notificaciones por email automáticas al confirmar o cancelar una reserva (Spring Mail).
 - API REST completa para una posible aplicación móvil (Android/iOS).
 - Panel de estadísticas avanzado con gráficas de ocupación semanal (Chart.js).
 - Despliegue en la nube (Railway o Render) con variables de entorno para las credenciales.
 - Sistema de valoraciones: los socios puntúan las clases asistidas.
-

11. Glosario

- **BCrypt**: algoritmo de hashing para contraseñas con salt aleatorio, resistente a fuerza bruta.
- **CRUD**: Create, Read, Update, Delete — operaciones básicas sobre datos persistentes.
- **JPA**: Java Persistence API — especificación para mapear objetos Java a tablas de BD relacional.
- **JPQL**: lenguaje de consultas de JPA orientado a objetos, similar a SQL pero sobre entidades.
- **MVC**: Model-View-Controller — patrón que separa datos, presentación y flujo de control.
- **Spring Boot**: framework Java que simplifica la configuración y arranque de aplicaciones Spring.
- **Spring Security**: módulo para autenticación, autorización y protección contra ataques web.

- **Thymeleaf:** motor de plantillas Java para generar HTML en servidor con integración Spring MVC.
 - **Transacción ACID:** operación de BD que garantiza Atomicidad, Consistencia, Aislamiento y Durabilidad.
-

12. Bibliografía y webgrafía

- Spring Boot Documentation (3.2):
<https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/>
- Spring Security Reference (6.x): <https://docs.spring.io/spring-security/reference/>
- Spring Data JPA Reference:
<https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
- Thymeleaf Documentation:
<https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>
- OpenPDF GitHub: <https://github.com/LibrePDF/OpenPDF>
- Apache Commons CSV: <https://commons.apache.org/proper/commons-csv/>
- MySQL 8.0 Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>
- Bootstrap Icons: <https://icons.getbootstrap.com/>
- Baeldung Spring Security Tutorials: <https://www.baeldung.com/security-spring>
- JUnit 5 User Guide: <https://junit.org/junit5/docs/current/user-guide/>